

1. Selectors

CSS selectors are used to target HTML elements so that styles can be applied to them. A selector tells the browser which element(s) should receive the specified CSS rules.

Types of Selectors

a) Element Selector

Selects all elements of a specific HTML tag.

Syntax:

CSS

```
p {  
    text-align: left;  
    font-size: 0.8em;  
    letter-spacing: 0.05em;  
    text-transform: uppercase;  
    color: var(--ink-soft);  
    padding: 0 10px 8px 0;  
    border-bottom: 1px solid var(--line);  
}
```

All <p> elements on the webpage will have blue text.

b) Class Selector

Selects elements with a specific class attribute.

Syntax:

CSS

```
.wrap {  
    max-width: 900px;  
    margin: 40px auto;  
    padding: 24px;  
    background: var(--panel);  
    border-radius: 16px;  
}
```

HTML

```
<div class="wrap">Content</div>
```

Only elements with class="box" receive the style.

c) ID Selector

Selects one unique element with a specific ID.

Syntax:

CSS

```
#header {  
    background-color: black;  
}
```

HTML

```
<header id="header"> </header>
```

An ID should be unique and used only once per page.

d) Universal Selector

Selects every element.

```
* {  
    margin: 0;  
    padding: 0;  
}
```

Commonly used for CSS reset.

e) Group Selector

Applies the same styles to multiple elements.

```
h1, h2, h3 {  
    color: navy;  
}
```

f) Descendant Selector

Targets elements inside another element.

```
div p {  
    color: red;  
}
```

Only <p> elements inside a <div> are selected.

i) Pseudo-class Selector

```
a:hover {  
    color: red;  
}
```

Applies styles when an element is in a particular state.

Examples:

:hover

:focus

:active
:visited

2. Colors

The color property changes text color, while background-color changes the background.

color: red;

background: blue;

Different ways

- **Hexadecimal**
- **RGB**
- **RGBA**
- **HSL**

Hexadecimal

color: #ff0000;

Format:

#RRGGBB

Example:

- Red → #ff0000
- Green → #00ff00
- Blue → #0000ff

RGB

color: rgb(255,0,0);

Each value ranges from 0–255.

RGBA

color: rgba(255,0,0,0.5);

The last value controls transparency.

Range:

0 = Transparent

1 = Fully Visible

HSL

color: hsl(120,100%,50%);

Represents:

- Hue
- Saturation
- Lightness

3. Units

CSS units define the size of text, spacing, width, height, and other measurements.

—Absolute Units

Unit	Meaning
px	Pixels
pt	Points
cm	Centimeters
mm	Millimeters
in	Inches

Example

font-size: 18px;

—Relative Units

em

Relative to the parent element's font size.

font-size: 2em;

If parent font size is 16px

2em = 32px

rem

Relative to the root (html) font size.

font-size: 2rem;

If root font size is 16px

2rem = 32px

%

Percentage relative to the parent.

width: 50%;

vw

Viewport Width

width: 100vw;
100% of browser width.

vh
Viewport Height
height: 100vh;
100% of browser height.

4.Fonts

CSS fonts determine the appearance of text.

font-family

Specifies the font.

font-family: Arial, sans-serif;

font-size

Defines text size.

font-size: 18px;

font-weight

Controls thickness.

font-weight: bold;

Common values

- 100
- 300
- 400 (Normal)
- 700 (Bold)
- 900

font-style

font-style: italic;

Values

- normal

- italic
- oblique

text-align

text-align: center;

Values

- left
- right
- center
- justify

text-decoration

text-decoration: underline;

Examples

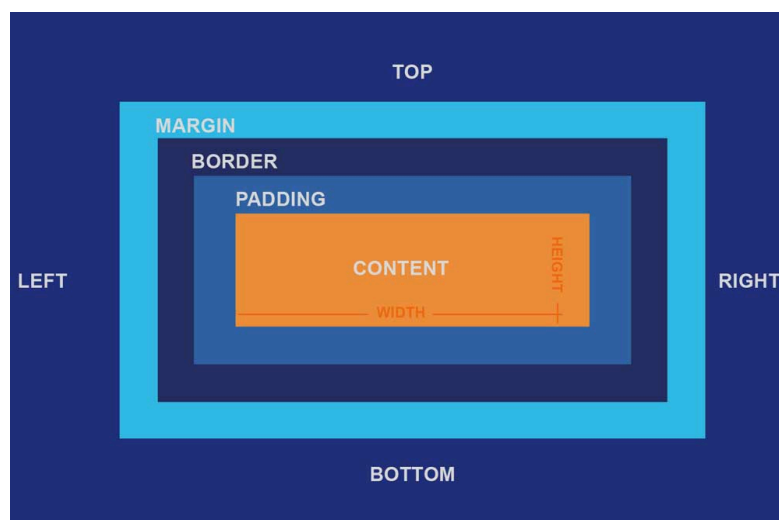
- underline
- overline
- line-through
- none

line-height

Controls spacing between text lines.

line-height: 1.5;

5.Box Model



The CSS Box Model is a layout model that describes how the browser calculates the size and spacing of every HTML element. According to the box model, each element is represented as a rectangular box consisting of four layers: content, padding, border, and margin. These layers together determine the total space occupied by an element on a web page.

Components of the Box Model

1. Content – The actual text, image, or other content displayed inside the element.
2. Padding – The space between the content and the border.
3. Border – The outline that surrounds the padding and content.
4. Margin – The transparent space outside the border that separates the element from neighboring elements.

6.Margin

Margin is the transparent space outside an element's border. It is used to create space between an element and the surrounding elements, helping to separate and organize the layout.

- Lies outside the border.
- Creates space between neighboring elements.
- Does not affect the element's background color.
- Can be set individually for each side or using shorthand.

Syntax

```
margin: 20px;
```

Individual Properties

```
margin-top: 10px;  
margin-right: 20px;  
margin-bottom: 15px;  
margin-left: 25px;
```

Shorthand Examples

margin: 10px;	All sides
margin: 10px 20px;	Top & Bottom Left & Right
margin: 10px 20px 30px 40px;	Top Right Bottom Left

7.Padding

Padding is the space between an element's content and its border. It provides internal spacing, making the content appear farther from the border.

Key Points

- Lies inside the border.
- Increases the space around the content.
- The element's background color extends into the padding area.
- Can be set individually or using shorthand.

Syntax

```
padding: 20px;
```

Individual Properties

```
padding-top: 10px;  
padding-right: 20px;  
padding-bottom: 15px;  
padding-left: 25px;
```

Shorthand Examples

```
padding: 10px;  
padding: 10px 20px;  
padding: 10px 20px 30px 40px;
```

8.Border

A border is the visible line that surrounds an element's content and padding. It defines the boundary of an element and can be customized in terms of width, style, and color.

Key Points

- Lies between the padding and margin.
- Can have different widths, styles, and colors.
- Supports rounded corners using border-radius.

Basic Syntax

```
border: 2px solid black;
```

Individual Properties

```
border-width: 2px;
```

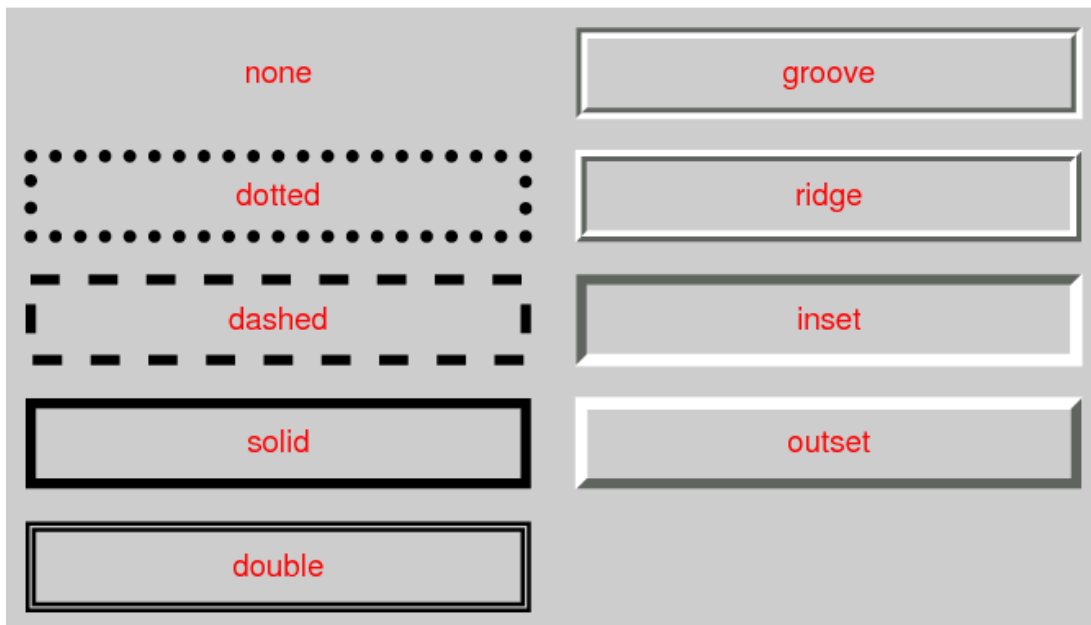

border-style: solid;
border-color: black;

Common Border Styles

- solid
- dashed
- dotted
- double
- groove
- ridge
- inset
- outset
- none

Rounded Corners
border-radius: 10px;

Circle
border-radius: 50%;



9.Display

1. display: block

Definition

A block-level element starts on a new line and occupies the full available width of its parent container by default.

Characteristics

- Starts on a new line.
- Takes up the full width available.
- Width and height can be set.
- Margins and padding work on all sides.

Examples of Block Elements

- <div>
- <p>
- <h1> to <h6>
- <section>
- <article>

Example

display: block;

2. display: inline

Definition

An inline element only occupies the width required by its content and does not start on a new line.

Characteristics

- Does not start on a new line.
- Occupies only the required width.
- Width and height cannot be explicitly set.
- Left and right margins/padding work normally.

Examples of Inline Elements

-
- <a>
-

- `<em>`

Example

`display: inline;`

3. display: inline-block

Definition

An inline-block element behaves like an inline element but allows width and height to be specified.

Characteristics

- Appears on the same line as other inline elements.
- Width and height can be set.
- Margins and padding work on all sides.

Example

`display: inline-block;`

4. display: none

Definition

Removes an element completely from the document layout. The element is not displayed and does not occupy any space.

Characteristics

- Element is completely hidden.
- No space is reserved in the layout.
- Often used to hide elements dynamically with CSS or JavaScript.

Example

`display: none;`

5. display: flex

Definition

Creates a flex container that uses the Flexbox Layout Model to arrange child elements in a single row or column.

Characteristics

- One-dimensional layout (row or column).
- Easy alignment and spacing of items.
- Responsive layout support.

Example

```
.container {  
  display: flex;  
}
```

6. display: grid

Definition

Creates a grid container that uses the CSS Grid Layout Model to arrange child elements in rows and columns.

Characteristics

- Two-dimensional layout.
- Supports both rows and columns simultaneously.
- Ideal for complex page layouts.

Example

```
.container {  
  display: grid;  
}
```

10.Position

The position property in CSS specifies how an element is positioned within a webpage. It determines the positioning method used for an element and controls how properties such as top, right, bottom, and left affect its placement.

Types of Position

1. position: Static

Static is the default positioning for all HTML elements. Elements are displayed in the normal document flow, and the top, right, bottom, and left properties have no effect.

Characteristics

- Default position.
- Follows the normal flow of the document.
- Offset properties (top, left, etc.) do not work.

Example

```
.box {  
  position: static;  
}
```

2. position: relative

An element with `position: relative;` is positioned relative to its normal position in the document flow.

Characteristics

- Remains in the normal document flow.
- Original space is preserved.
- Can be shifted relative to its original position.
- Often used as a reference for absolutely positioned child elements.

Example

```
.box {  
  position: relative;  
  top: 20px;  
  left: 30px;  
}
```

Result:

The element moves 20px downward and 30px to the right, but its original space remains reserved.

3. position: absolute

Definition

An absolutely positioned element is removed from the normal document flow and is positioned relative to its nearest positioned ancestor (an ancestor with `position: relative`, `absolute`, `fixed`, or `sticky`). If no positioned ancestor exists, it is positioned relative to the initial containing block (the page).

Characteristics

- Removed from the normal document flow.
- Does not reserve its original space.

- Positioned using top, right, bottom, and left.
- Commonly used for overlays, tooltips, and badges.

Example

```
.parent {  
  position: relative;  
}  
  
.child {  
  position: absolute;  
  top: 10px;  
  right: 10px;  
}
```

Result:

The child element is positioned 10px from the top and 10px from the right of its parent.

4. position: fixed

Definition

A fixed positioned element is removed from the normal document flow and positioned relative to the browser viewport. It remains in the same position even when the page is scrolled.

Characteristics

- Removed from the normal document flow.
- Stays fixed while scrolling.
- Positioned relative to the viewport.
- Commonly used for navigation bars and floating action buttons.

Example

```
.box {  
  position: fixed;  
  bottom: 20px;  
  right: 20px;  
}
```

Result:

The element remains 20px from the bottom and 20px from the right of the browser window, regardless of scrolling.

5. position: sticky

Definition

A sticky positioned element behaves like a relatively positioned element until it reaches a specified scroll position. It then becomes fixed within its container until the container is no longer visible.

Characteristics

- Combines the behavior of relative and fixed.
- Requires an offset value (e.g., top: 0).
- Commonly used for sticky headers and navigation bars.

Example

```
.header {  
  position: sticky;  
  top: 0;  
}
```

Result:

The header scrolls normally until it reaches the top of the viewport, where it remains fixed while the rest of the page continues to scroll.

To Learn More about positioning :

 [Learn CSS Positions in 4 minutes](#)